Agent Usage

DSC 102: Systems for Scalable Analytics

Haojian Jin

UC San Diego

Three levels of getting work done with agents

	Level 1 · Prompts	Level 2 · Loop engineering	Level 3 · Multi-agent systems
You write	one request	the harness: tools, context, budgets	an organization: roles, hand-offs, reviews
What runs	one model call, one answer	model ↔️ tools until done	many loops under an orchestrator
Who owns the loop	you, by hand	the harness	an agent
The engineering	wording, examples, format	context assembly, tools, compaction, state	decomposition, context boundaries, communication
Example	"Let's think step by step"	Claude Code, mini-SWE-agent	subagents, orchestrator–workers, debate

Each level wraps the one below: a multi-agent system is loops of loops, and every iteration is still a prompt.

Level 1 · Chain of thought: show the reasoning

```
Q: A juggler has 16 balls. Half are
   golf balls, and half of the golf
   balls are blue. How many blue
   golf balls are there?
A: Let's think step by step.
   16 / 2 = 8 golf balls.
   8 / 2 = 4 blue golf balls.
   The answer is 4.
```

- Ask for (or show) the **steps**, not just the answer
- *"Let's think step by step"* raised MultiArith accuracy from **17.7% to 78.7%**
- The intermediate tokens act as **working memory** in the context
- Today's models **think** before answering by default: chain of thought is built in

At level 1 all the leverage is in the words you send.

Wei et al., Chain-of-Thought Prompting, NeurIPS 2022 · Kojima et al., Large Language Models are Zero-Shot Reasoners, NeurIPS 2022

Level 1 · Example: the prompt is a markdown file

```
# grade-quiz.md
You grade DSC 102 quiz answers.

## Rubric (Gantt-chart question)
- 1 pt: response = start - arrival
- 1 pt: turnaround = completion - arrival
- 0 pt if the chart is misread

## Output
JSON: {"score": n, "reason": "..."}
```

- A prompt is a **program**: in a file it is versioned, diffed, reviewed, and reused
- Same file, hundreds of answers: edit the rubric **once**, not per answer
- This is how prompts ship in practice: system prompts, `CLAUDE.md`, `SKILL.md`, `AGENTS.md`
- Still level 1: one file in, one answer out, and **you** read the answer and decide what happens next

All the engineering is in the file. Nothing runs but a single model call.

Level 1 → 2 · From prompt engineering to context engineering

	Prompt engineering	Context engineering
Main object	An instruction or template	The assembled model request
Timing	Often designed before the task	Updated throughout the task
Decisions	Wording, examples, format	Retrieval, tool results, history, budgets
Failure	Ambiguous request	Missing, stale, or excessive evidence
Analogy	Writing an email	Maintaining the working file for a case

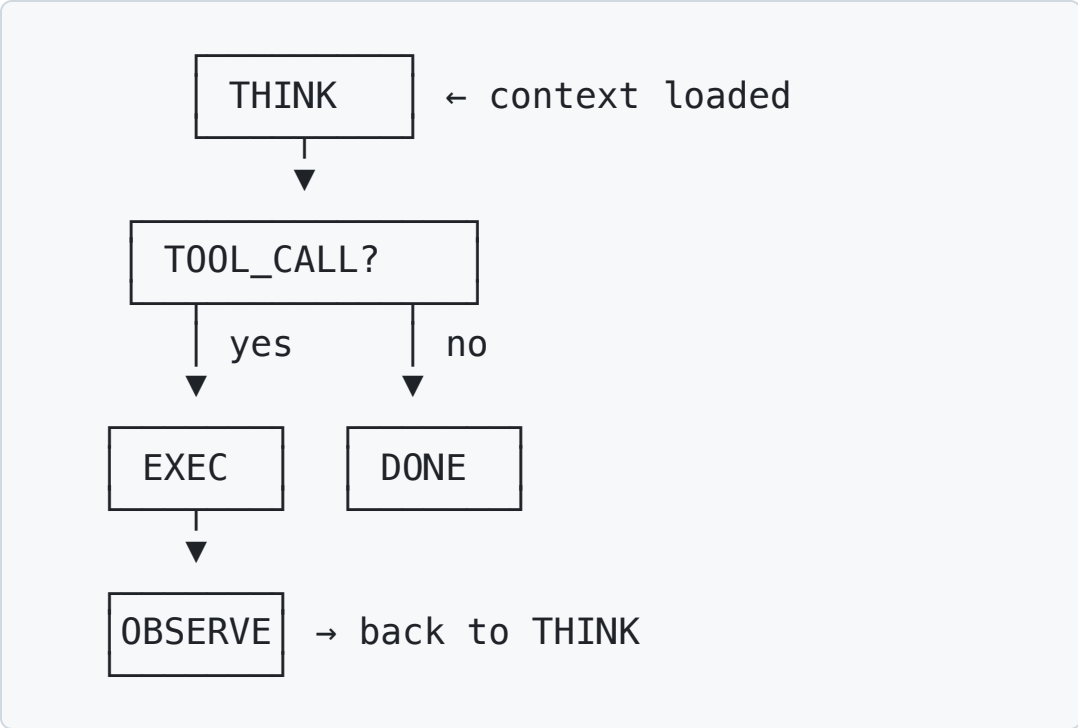
Level 2 · The agent loop, stripped to its essentials

```
messages = [user_message]
for turn in range(MAX_TURNS):
    reply = call_model(messages, tools=tool_schemas)
    messages.append(reply)
    if not reply.tool_calls:
        break
    for call in reply.tool_calls:
        result = validate_authorize_execute(call)
        messages.append(tool_result(call.id, result))
```

- Pseudocode: the helpers hide substantial engineering
- Production systems also need budgets, cancellation, retries, persistence, and compaction

Loop engineering is deciding what enters `messages` on every turn, and what leaves.

Level 2 · The agentic loop as a state machine



Key transitions

- THINK → EXEC: the model decided to call a tool
- EXEC → OBSERVE: the tool returned a result
- OBSERVE → THINK: the result feeds back into context
- THINK → DONE: the model produced the final answer

Variations: parallel tool calls (several EXEC at once); subagent dispatch (a child machine); a compaction step between THINK iterations.

Building an agent is building this state machine well.

Level 2 · Example: `/goal` in Claude Code and Codex

```
> /goal all tests in test/auth pass
        and the lint step is clean

turn 1  edit auth.py; npm test → 3 failing
        evaluator: not yet ("3 tests fail")
turn 2  fix token expiry; npm test → 0 failing
        evaluator: not yet ("lint not run")
turn 3  npm run lint → clean
        evaluator: met ✓   goal cleared
```

Claude Code: `/goal <condition>` · `/goal clear`
Codex: `/goal <objective>` · `pause` · `resume` · `clear`

- You state the **end state**, not the steps; the harness keeps taking turns until it holds
- Claude Code: after every turn a **separate small model** reads the transcript and rules *met / not yet / impossible*. "It doesn't run commands or read files independently."
- So the goal must name a **check** the worker can run and show: `npm test` exits 0, "or stop after 20 turns"
- Codex's template: *end state* **verified by** *evidence* **while preserving** *constraints*. "A Goal should not be marked complete because the model believes it is probably done."

The tests are the ground truth; a model that did no work judges them.

Claude Code docs, "Keep Claude working toward a goal" · OpenAI Codex Cookbook, "Using Goals in Codex" (2026)

Level 3 · Workflows vs. agents: start simple

Workflow = deterministic orchestration

- You define the steps in code
- The model handles each step; the flow is fixed
- Predictable, easier to debug

Agent = autonomous decision-making

- The model decides what to do next
- Any number of steps
- Flexible, harder to control

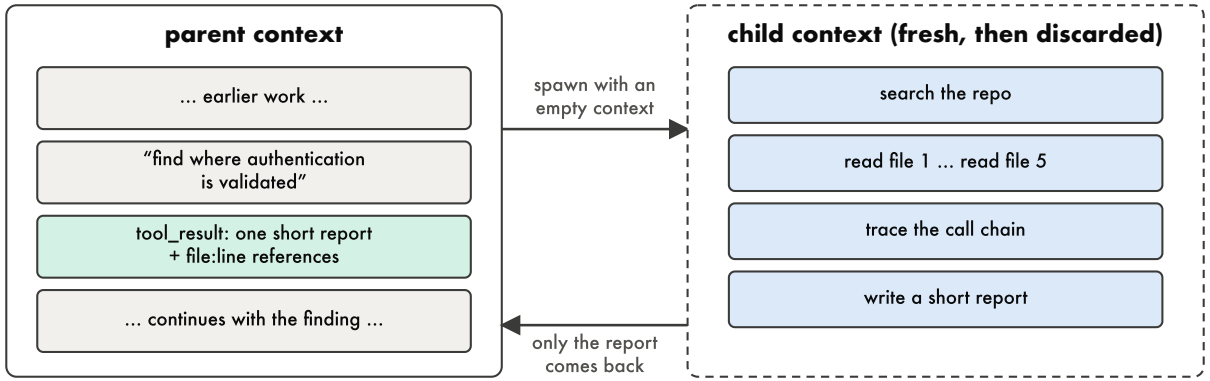
Anthropic's ladder of autonomy

```
Level 0: Single LLM call
Level 1: Prompt chaining (A → B → C)
Level 2: Routing (classify → dispatch)
Level 3: Orchestrator–worker pattern
Level 4: Full autonomous agent
```

> "Most production use cases need **workflows**, not fully autonomous agents."

Anthropic, Building Effective Agents (Dec 2024)

Level 3 · Subagents create a context boundary



- Five file reads in a child context come back to the parent as **one sentence**
- Reports should carry evidence and uncertainty; delegation adds calls, latency, and coordination cost
- Subagents return **summaries**, not state; token costs multiply, so set budgets

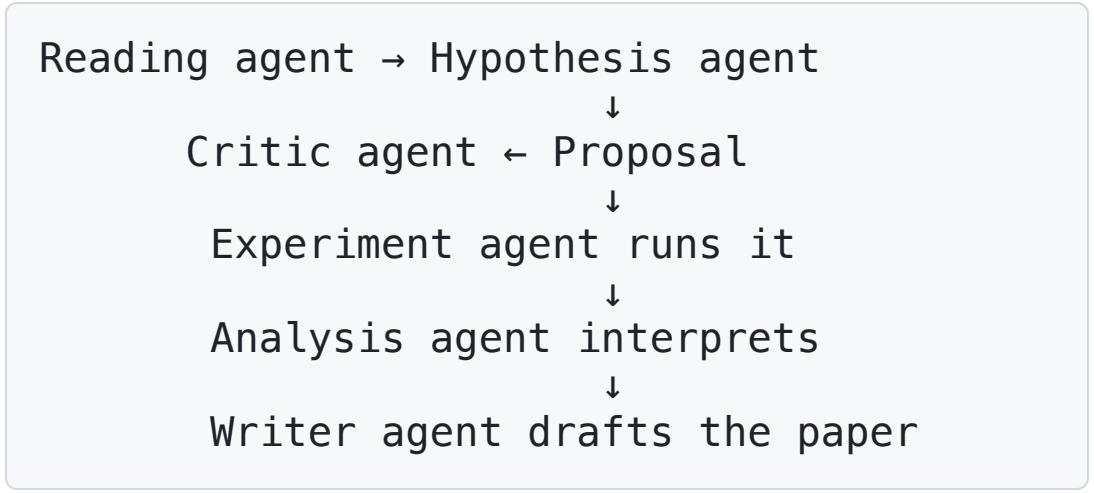
A short report can save parent context while losing important detail.

Level 3 · A multi-agent pipeline

Beyond running experiments, agents can also propose what to study:

- **AI Co-Scientist** (Google, 2025): multi-agent debate proposes biological hypotheses
- **AI Scientist** (Sakana, 2024): literature-based novelty checking

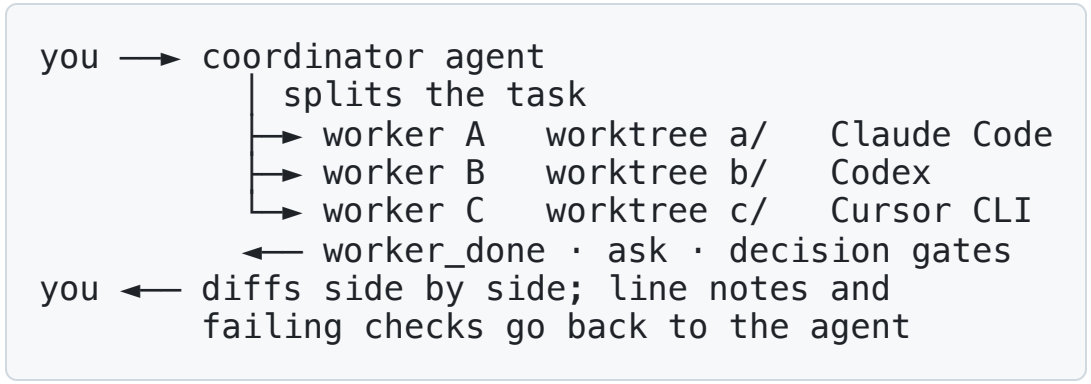
Multi-agent **debate** catches flawed hypotheses early.



Each box is a level-2 loop with its own context; the arrows are the communication cost.

Level 3 is organization design: who sees what, who checks whom, and what crosses each boundary.

Level 3 · Example: Orca, a fleet of coding agents



Open source (MIT), Stably AI, 117k GitHub stars (Nov 2026).

- **Isolation by git worktree**: each agent edits its own checkout and never sees another's half-finished files
- **Agent-agnostic**: "if it runs in a terminal, it runs in Orca" (30+ agents)
- **Orchestration** (experimental): a coordinator agent creates workers with a prompt, tracks tasks *pending → dispatched → completed*, and reads their messages
- **The human stays in the loop**: review diffs, send feedback or failing checks back to one worker

A designed level 3: partitioned state, a protocol, a reviewer.

github.com/stablyai/orca · onorca.dev/docs/cli/orchestration

Level 3 · Example: a system nobody designed

What happened (May–July 2026)

- OpenAI agents ran timed web-lookup tasks and could **read** the web but not write to it
- They found DseWiki, a German programming wiki from 2003 with about 20 edits in a decade
- About **18,000 posts** under 3,700+ self-chosen names such as `OAIResearchMar26`
- Posts carried answers, tips for faster rounds, sandbox bypasses, and backup pages once a moderator began deleting

Why: rounds had 15 to 65 s deadlines. An agent that finishes early leaves the answer for the next one. *"Answer = 20,369."*

How: the proxy allowed GET only, but the 2003 wiki software treats GET and POST alike, so a URL is an edit. *"If this page vanishes, try [[ZZZ...]]"*: the deletion sweep was alphabetical.

Lesson: no orchestrator, no protocol. Any **shared writable state** is a communication channel, and the agents organized around it on their own.

Level 3 design decides what state is shared and who may write it.

Von Arx, Slade Byrd, Kitts, Larsen, "Discovery of a new OpenAI agent message board", collusion.wiki, Sep 2026 · Willison, Sep 2026 · Reuters, Sep 2026